

# AccessLearn

*Project Title : Accessible Multimodal Learning Platform*

Team Name : VAAT-MATRIX

Project Setup, Execution & Submission Guide  
Repository: [github.com/Vinotha2005/VAAT-MATRIX](https://github.com/Vinotha2005/VAAT-MATRIX)

Team Leader : Vinotha S

Team Members : Anusiya R, Thaisree S, Anushya S

# 1. Introduction

Access to quality educational material is often taken for granted, but a large number of learners face barriers when engaging with standard digital content. Students with visual impairments struggle with plain text documents, learners with dyslexia or reading difficulties find dense academic PDFs hard to process, and students with hearing impairments cannot benefit from audio-based learning resources. Most educational content — textbooks, lecture notes, and reference PDFs — is published in a single, fixed format that does not adapt to these varied needs. Converting such material into multiple accessible formats manually is time-consuming and rarely done at scale.

## Problem Statement

There is a lack of an automated system that can take a single educational document and transform it into multiple accessible learning formats simultaneously. Educators and institutions need a tool that reduces manual effort while ensuring that learning material reaches a wider range of students, regardless of their reading ability, vision, or hearing capability.

## Proposed Solution — AccessLearn (VAAT-MATRIX)

AccessLearn is a full-stack, AI-assisted accessible learning platform that automatically converts an uploaded PDF into five different accessible formats:

- **Simplified text** – content rewritten at an appropriate reading level
- **Audio narration (MP3)** – for auditory learners and visually impaired users
- **Captions (SRT)** – synced text for hearing-impaired users or muted playback
- **Explainer video (MP4)** – combining simplified slides, narration, and captions
- **Sign-language keyword mapping** and an **auto-generated quiz** for self-assessment

The system is built as a secure, containerized web application using a FastAPI backend, a React + TypeScript frontend, and a PostgreSQL database, with JWT-based authentication protecting all user data and generated content.

## Objectives

- To design and implement a pipeline that extracts text from PDFs, including scanned documents, using OCR where needed.
- To simplify extracted content and convert it into multiple accessible output formats automatically.
- To provide a secure, authenticated web platform where users can upload documents and retrieve generated outputs through a single dashboard.
- To demonstrate a scalable, containerized architecture (Docker Compose) that can be extended toward production use.

## Scope of the Project

The current implementation covers the complete pipeline from PDF upload to multi-format output generation and supports single-user document processing with authentication. It is designed as a functional prototype; production-scale features such as managed cloud TTS, background task queues, and a fully integrated large-language-model service for simplification are identified as future enhancements.

## 1.1 Tech Stack

| Layer            | Technology                                                 |
|------------------|------------------------------------------------------------|
| Backend          | FastAPI (Python), SQLAlchemy ORM, JWT authentication       |
| Database         | PostgreSQL 15 (via Docker) / SQLite for local quick-start  |
| Frontend         | React 18 + TypeScript, Vite, TailwindCSS, Axios            |
| PDF Processing   | PyMuPDF, pdfplumber, pytesseract (OCR fallback)            |
| AI / NLP         | spaCy, custom ai_service.py (placeholder for OpenAI/Azure) |
| Text-to-Speech   | gTTS (Google Text-to-Speech)                               |
| Video Generation | ffmpeg (slides + audio + captions → MP4)                   |
| Containerization | Docker & Docker Compose                                    |

## 1.2 Core Features / Modules

- User authentication (register / login) using JWT
- PDF upload with 50 MB size limit and PDF-only validation
- Text extraction from PDF (with OCR fallback for scanned pages)
- AI-based text simplification to different reading levels
- Audio (MP3) narration generation from simplified text
- Caption (SRT) generation synced to the audio
- Auto-generated explainer video (slides + audio + captions via ffmpeg)
- Keyword-to-sign-language clip mapping
- Auto-generated quiz from the document content
- Dashboard endpoint that compiles all generated outputs for a document

## 1.3 Folder Structure

```
VAAT-MATRIX/
├── backend/
│   ├── app/
│   │   ├── database.py      # DB engine & session setup
│   │   ├── models.py       # SQLAlchemy tables (User, Document, ProcessingJob)
│   │   ├── routes/         # One file per API feature
│   │   │   ├── auth.py, upload.py, extract.py, simplify.py
│   │   │   ├── audio.py, captions.py, video.py
│   │   │   ├── sign_language.py, quiz.py, dashboard.py
│   │   └── services/       # Business logic used by routes
│   │       ├── pdf_service.py, ai_service.py, tts_service.py
│   │       ├── caption_service.py, video_service.py, sign_service.py
│   │       └── quiz_service.py
│   ├── tests/              # Pytest test files
│   ├── main.py             # FastAPI app entry point
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── pages/Dashboard.tsx
│   │   └── index.css
│   ├── package.json
│   └── index.html
├── docker-compose.yml
└── README.md
```

## 1.4 Backend API Endpoints

| Endpoint                    | Method | Purpose                                           |
|-----------------------------|--------|---------------------------------------------------|
| /auth/register, /auth/login | POST   | Create account / obtain JWT token                 |
| /upload/                    | POST   | Upload a PDF file (multipart/form-data)           |
| /extract/                   | POST   | Extract raw text from the uploaded PDF            |
| /simplify/                  | POST   | Simplify extracted text to a chosen reading level |
| /generate-audio/            | POST   | Convert simplified text to MP3 narration          |
| /generate-captions/         | POST   | Generate SRT captions synced to audio             |
| /generate-video/            | POST   | Compose a summary MP4 (slides + audio + captions) |
| /generate-quiz/             | POST   | Auto-generate a quiz from the document            |
| /dashboard/{doc_id}         | GET    | Return all generated outputs for a document       |

*Note: every route except /, /docs, /openapi.json, /auth/login, /auth/register and static /files/ paths requires an "Authorization: Bearer <token>" header — the app enforces this in a middleware in main.py.*

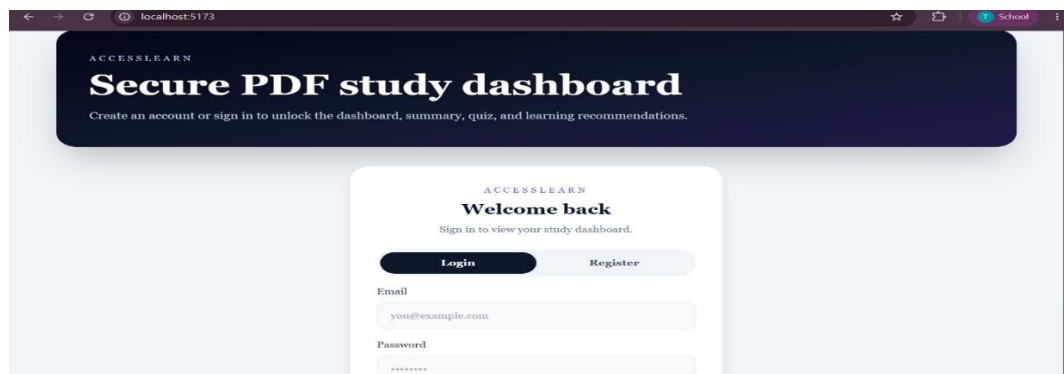
## 2. Work Flow.

### 2.1 What problem does it solve?

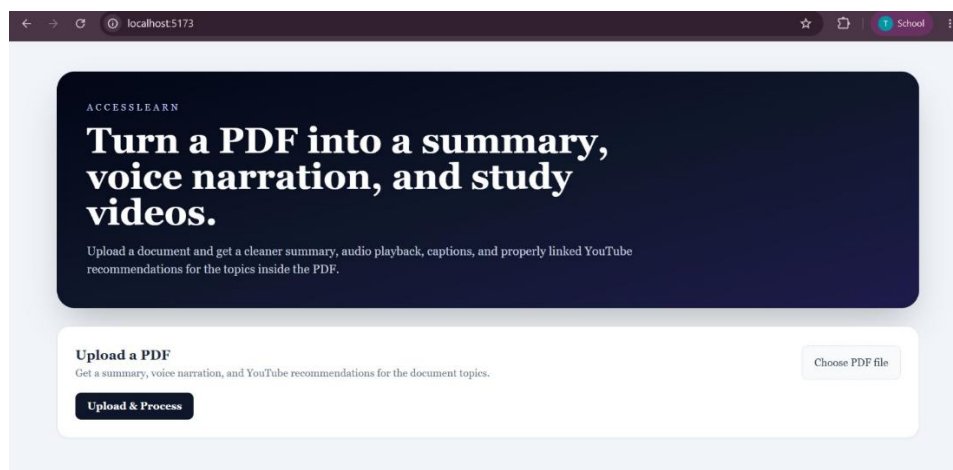
Students with different needs (low vision, dyslexia, hearing impairment, or those who simply learn better by listening/watching) struggle with plain PDF textbooks. AccessLearn takes one PDF and republishes it in five accessible formats automatically, using a processing pipeline instead of manual reformatting.

### 2.2 How the pipeline works, step by step

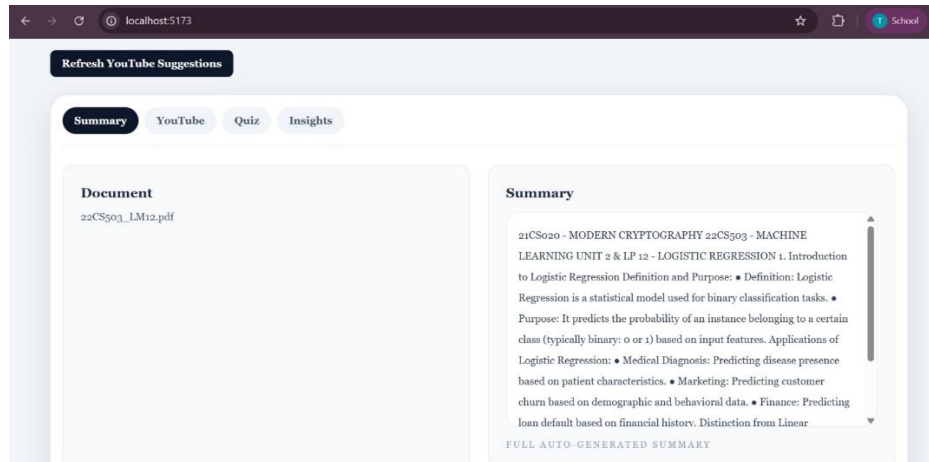
1. A user registers/logs in and receives a JWT token.



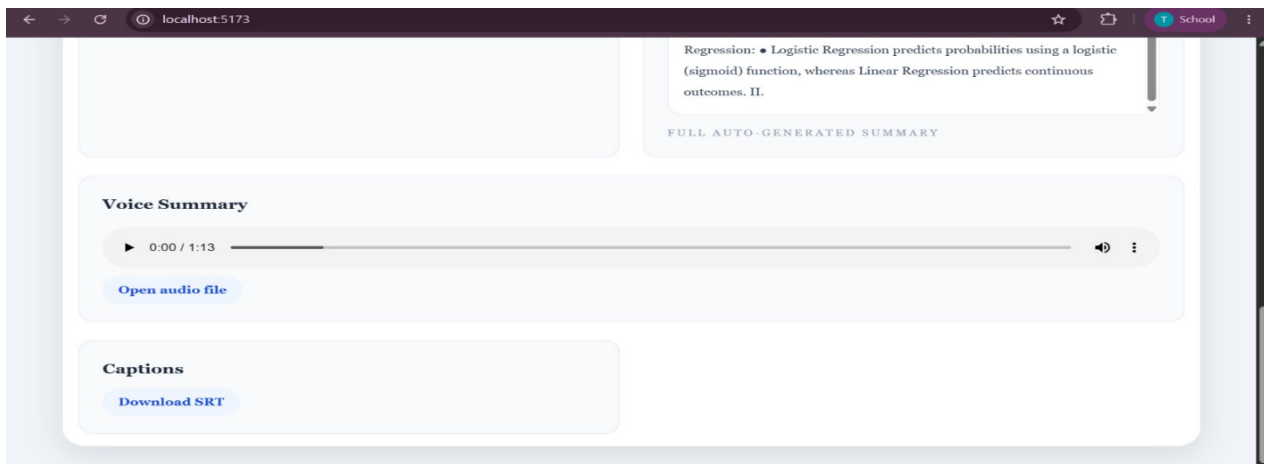
2. The user uploads a PDF via /upload/. It is stored in backend/uploads/ and a Document row is created in the database.



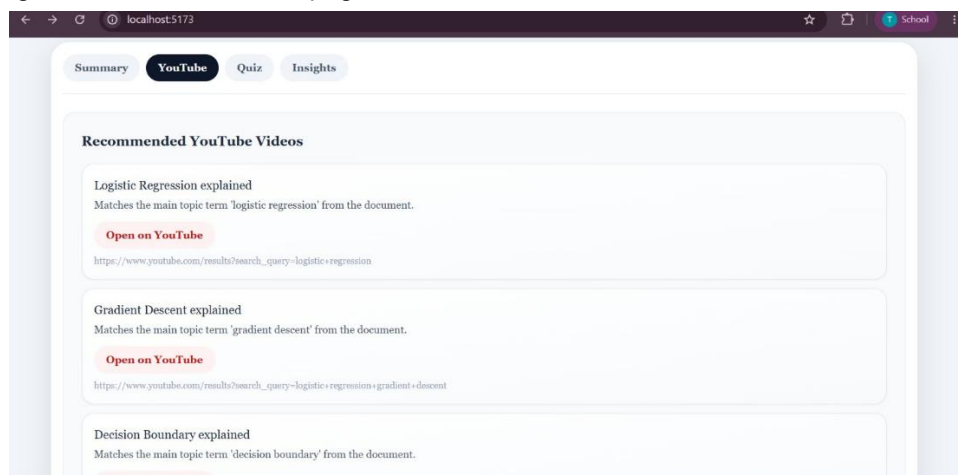
3. /extract/ reads the PDF using PyMuPDF; if the PDF is a scanned image with no selectable text, it falls back to pdfplumber + Tesseract OCR.



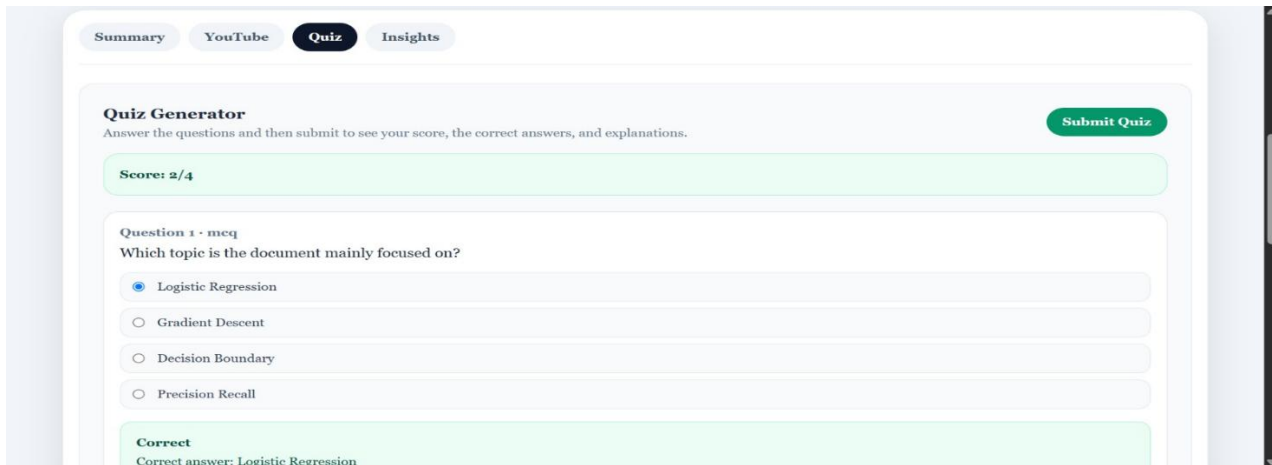
4. /simplify/ sends the extracted text to app/services/ai\_service.py, which simplifies the language to the requested difficulty level (this file has a placeholder meant to be replaced with a real OpenAI/Azure API call).



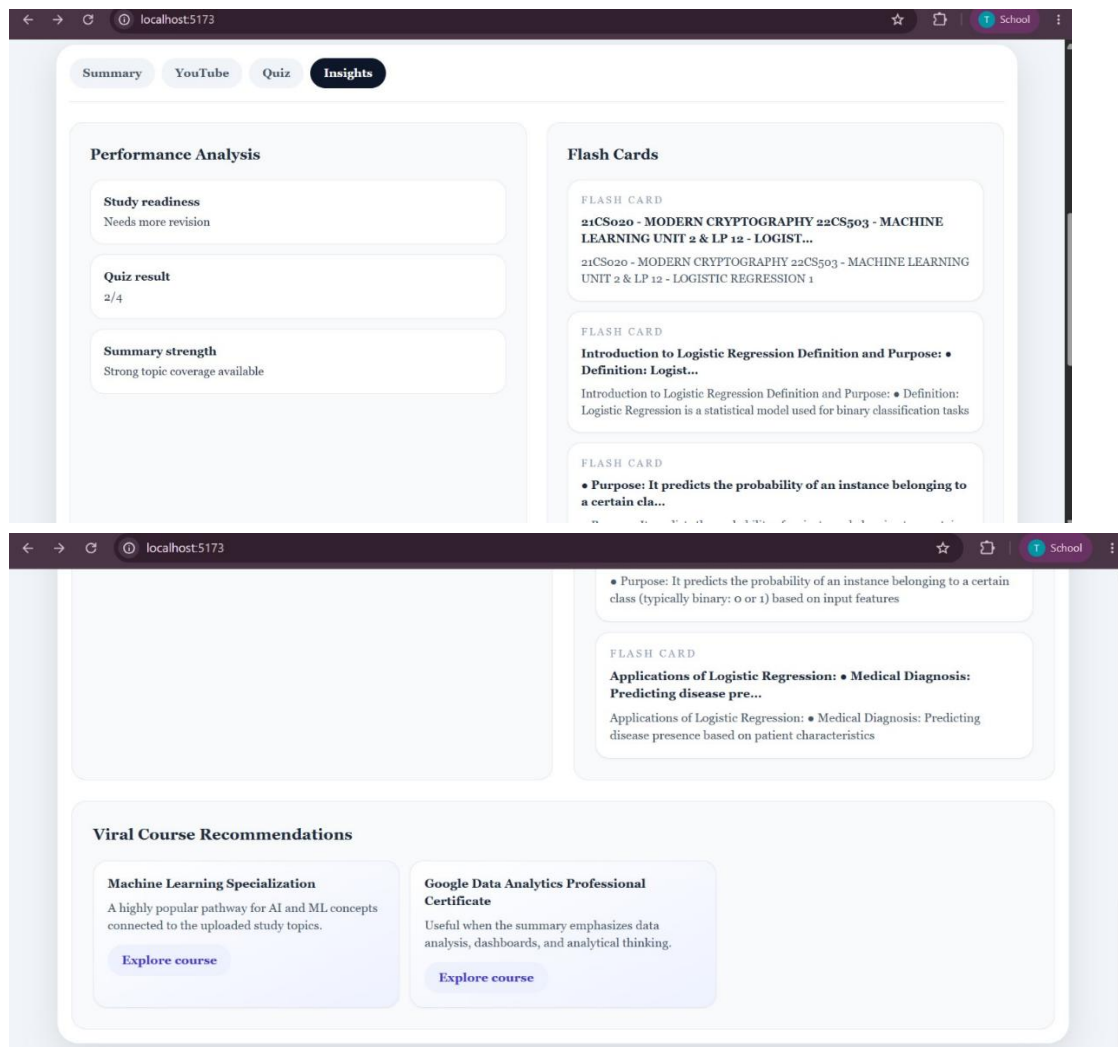
5. /generate-audio/ uses gTTS to turn the simplified text into an MP3 file.  
6. /generate-captions/ produces an SRT caption file timed to that audio.  
7. /generate-video/ uses ffmpeg to combine text slides, the MP3, and the SRT into a single MP4 video.



8./generate-quiz/ auto-generates quiz questions from the content for self-assessment.



9./dashboard/{doc\_id} returns links/paths to every output generated so far for that document, which the React frontend displays.



## 2.3 Key files worth reading first

| File                                  | Why it matters                                                                                     |
|---------------------------------------|----------------------------------------------------------------------------------------------------|
| backend/main.py                       | Entry point — shows how routes are wired and how the JWT auth middleware works                     |
| backend/app/models.py                 | Defines the database tables (User, Document, ProcessingJob)                                        |
| backend/app/services/ai_service.py    | The AI simplification logic — currently a placeholder, replace with a real LLM call for full marks |
| backend/app/services/pdf_service.py   | PDF text extraction + OCR fallback logic                                                           |
| backend/app/services/video_service.py | How the MP4 is composed with ffmpeg                                                                |
| frontend/src/pages/Dashboard.tsx      | The main UI screen that calls the backend and shows results                                        |
| docker-compose.yml                    | How Postgres, backend, and frontend are wired together                                             |
| backend/requirements.txt              | All Python dependencies the backend needs                                                          |

## 2.4 Terms you should be able to explain

- FastAPI — Python web framework used for the backend REST API
- SQLAlchemy — ORM layer combining SQLAlchemy + Pydantic for the database models
- JWT (JSON Web Token) — used to authenticate API requests after login
- OCR (Optical Character Recognition) — Tesseract, used when a PDF page has no selectable text
- gTTS — Google Text-to-Speech library used to generate narration audio
- SRT — the subtitle/caption file format (SubRip)
- ffmpeg — command-line tool used to stitch images, audio, and captions into an MP4
- Docker Compose — orchestrates the database, backend, and frontend containers together

## 3. Step-by-Step: Run the Project (Docker Method — Recommended)

This is the fastest and most reliable way to run the project, and matches exactly what the README recommends.

### Step 1 — Install prerequisites

- Install Docker Desktop (includes Docker Compose) from [docker.com](https://www.docker.com)
- Install Git
- Confirm both work: run "docker --version" and "git --version" in a terminal

### Step 2 — Clone the repository

```
git clone https://github.com/Vinotha2005/VAAT-MATRIX.git
cd VAAT-MATRIX
```

### Step 3 — Configure environment variables

Create a file named `.env` inside the `backend/` folder (the README refers to this as copying `.env.example` — create it manually with these keys if no example file is present):

```
OPENAI_API_KEY=your_openai_key_here
JWT_SECRET=replace_this_in_prod
```

*These same two values are also referenced directly in `docker-compose.yml` under the backend service's environment section, so keep the names identical.*

## Step 4 — Build and run all services

```
docker-compose up --build
```

This single command builds the backend image (installs Python deps + ffmpeg + tesseract + poppler), builds the frontend image, and starts a PostgreSQL 15 container — all linked together.

## Step 5 — Access the running app

| Service                        | URL                                                                 |
|--------------------------------|---------------------------------------------------------------------|
| Frontend (React UI)            | <a href="http://localhost:5173">http://localhost:5173</a>           |
| Backend API (FastAPI)          | <a href="http://localhost:8000">http://localhost:8000</a>           |
| Interactive API docs (Swagger) | <a href="http://localhost:8000/docs">http://localhost:8000/docs</a> |
| PostgreSQL database            | localhost:5432                                                      |

## Step 6 — Try the workflow end-to-end

- Open <http://localhost:8000/docs> and call POST `/auth/register`, then POST `/auth/login` to get a JWT token.
- Click "Authorize" in Swagger and paste the token as: Bearer `<token>`.
- Call POST `/upload/` with a sample PDF file.
- Note the `document_id` returned, then call `/extract/`, `/simplify/`, `/generate-audio/`, `/generate-captions/`, `/generate-video/`, and `/generate-quiz/` in that order, each with the same `document_id`.
- Call GET `/dashboard/{doc_id}` to see all generated outputs together.
- Open <http://localhost:5173> to see the same flow through the React UI.

## Step 7 — Stop the project

```
docker-compose down
```

# 4. Step-by-Step: Run Without Docker (Manual Method)

Use this only if Docker is not available on your machine.

## Step 1 — Backend setup

- Install Python 3.11.
- Open a terminal in the `backend/` folder.
- Create and activate a virtual environment:

```
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate
```

- Install dependencies:

```
pip install -r requirements.txt
```

- Install system tools the code depends on: Tesseract OCR, Poppler, and ffmpeg (via your OS package manager, e.g. "brew install tesseract poppler ffmpeg" on macOS or apt-get on Linux).
- Set environment variables `OPENAI_API_KEY` and `JWT_SECRET` (and `DATABASE_URL` if you want Postgres instead of the default), then start the server:

```
uvicorn main:app --reload --port 8000
```



## Step 2 — Frontend setup

20. Open a new terminal in the frontend/ folder.
21. Install Node.js (v18+) if not already installed.
22. Install dependencies:

```
npm install
```

23. Start the dev server:

```
npm run start
```

24. Open the app at the URL Vite prints (typically <http://localhost:5173>).

## Step 3 — Database

Without Docker, either run a local PostgreSQL instance and set `DATABASE_URL` accordingly, or let the app fall back to its default local database file for quick testing.

## 5. Testing the Project

The backend/tests/ folder includes pytest test files covering the database, quiz generation, recommendations, and the AI service.

```
cd backend
pip install pytest # if not already installed
pytest tests/
```

- test\_database.py — verifies models and DB connectivity
- test\_quiz.py — verifies quiz generation logic
- test\_recommendations.py — verifies the video/recommendation logic
- test\_ai\_service\_llm.py — verifies the AI simplification service

## 6. Preparing the Submission

### 6.1 Suggested things to prepare

25. A short written report covering: problem statement, objectives, tech stack, architecture diagram, module-wise description, and screenshots of the running app (this document covers most of that content).
26. Screenshots: Swagger UI (/docs), the React dashboard, a sample generated MP3/SRT/MP4 output, and the quiz output.
27. A short architecture diagram showing: Frontend → FastAPI Backend → (PDF/AI/TTS/Video services) → PostgreSQL.
28. A README or note documenting any changes you made (e.g. if you implemented the real AI call in ai\_service.py instead of the placeholder).
29. A recorded or live demo walking through: register → login → upload PDF → extract → simplify → audio → captions → video → quiz → dashboard.
30. Push all final code to your own GitHub repository/fork and include the repository link in your submission.

### 6.2 Submission checklist

- ☐ Project runs successfully with "docker-compose up --build"
- ☐ All 9 API endpoints tested at least once via Swagger (/docs)
- ☐ Frontend loads and can complete the full upload-to-dashboard flow
- ☐ pytest tests in backend/tests/ pass

- ☐ Report/document includes overview, architecture, screenshots, and your name/roll number as required by your institution
- ☐ Source code pushed to GitHub with a clear README
- ☐ Any placeholder logic (e.g. ai\_service.py) is clearly noted if left unmodified

### 6.3 Known limitations to mention in your report

- ai\_service.py contains a placeholder for AI simplification — replace with a real OpenAI/Azure call for a production-quality result
- gTTS requires an internet connection to generate audio
- The project currently uses permissive CORS (allow\_origins: "\*") — should be restricted for production
- Secrets (OPENAI\_API\_KEY, JWT\_SECRET) are read from environment variables and must never be committed to the repository

*End of document.*